

Manual Técnico

Descargador de YouTube Pro Video & MP3 con Normalización de Audio

Plataformas: Windows 11 · Fedora 44

Versión: 3.0

Fecha: 2026

Lenguaje: Python 3.10+

Tabla de Contenidos

#	Sección
1.	Arquitectura del sistema
2.	Requisitos técnicos
3.	Módulos y funciones
4.	Normalización de audio (LUFS)
5.	Sistema de notificaciones
6.	Cookies y autenticación
7.	Gestión de errores y logging
8.	Estructura de archivos

1. Arquitectura del Sistema

El sistema es un script Python monolítico que orquesta componentes externos (yt-dlp, FFmpeg, PowerShell / notify-send) mediante subprocess. La lógica se organiza en capas funcionales:

Capa	Responsabilidad
Interfaz	Menú interactivo, entrada de URLs, selección de calidad
Descarga	yt-dlp con cookies, reintentos, historial
Post-proceso	FFmpeg: conversión, metadatos, normalización LUFS
Notificación	PowerShell Toast (Win) / notify-send (Linux)
Sistema	Detección de SO, navegador, creación de iconos

2. Requisitos Técnicos

Componente	Versión mínima	Propósito
Python	3.10	Ejecución del script
yt-dlp	2024.01+	Extracción y descarga
FFmpeg	6.0	Conversión, normalización, merge
Deno / Node	Opcional	Runtime JS para YouTube
pywin32	Opcional (Win)	Acceso directo .lnk
reportlab	4.0	Generación de PDFs

3. Módulos y Funciones

detectar_so()

Devuelve 'windows', 'fedora' o 'debian' según /etc/os-release o platform.system().

detectar_navegador()

Busca perfiles de Firefox, Chrome, Edge, Brave o Chromium en rutas típicas.

detectar_runtime_js()

Localiza 'deno', 'node' o 'bun' en PATH para habilitar extracción completa de YouTube.

notificar(titulo, msg)

Envía Toast en Windows (PowerShell) o notify-send en Linux. No bloqueante.

normalizar_audio(ruta)

Ejecuta FFmpeg loudnorm en 2 pasadas para ajustar LUFS objetivo.

descargar_media(...)

Función principal: configura yt-dlp, descarga, postprocesa y notifica.

crear_icono_escritorio()

Genera .lnk en Windows o .desktop en Fedora.

4. Normalización de Audio (LUFS)

La normalización se realiza con el filtro **loudnorm** de FFmpeg en modo **dual-pass**, que mide primero y aplica después. Esto evita distorsión y garantiza un volumen consistente entre pistas.

Parámetro	Valor	Descripción
I (Integrated)	-16 LUFS	Estándar YouTube / Spotify / Apple Music
TP (True Peak)	-1.5 dB	Margen para evitar clipping en DACs
LRA	11 dB	Rango dinámico; 11 dB es natural para voz/música
Sample rate	48000 Hz	Estándar de audio profesional

Primera pasada: mide `input_i`, `input_tp`, `input_lra`, `input_thresh` y `target_offset`. FFmpeg emite estos valores como JSON en `stderr`.

Segunda pasada: aplica `loudnorm` con los valores medidos y `linear=true` para corrección lineal sin compresión adicional.

5. Sistema de Notificaciones

SO	Mecanismo	Comando
Windows 11	Toast nativo	<code>powershell -Command [ToastNotificationManager]</code>
Fedora 44	libnotify	<code>notify-send -a 'Descargador' -i download <título> <msg></code>

Las notificaciones se envían al finalizar toda la cola de descargas con el número de archivos y el tiempo total. Si falla, se registra en el log sin interrumpir el flujo principal.

6. Cookies y Autenticación

El script extrae cookies del navegador automáticamente mediante `cookiesfrombrowser` de `yt-dlp`. El orden de preferencia es: `firefox` → `chrome` → `edge` → `brave` → `chromium` → `opera`.

Nota importante: Chrome 127+ introdujo cifrado App-Bound que puede impedir la lectura de cookies desde procesos externos. Se recomienda Firefox o cerrar Chrome antes de ejecutar el script.

7. Gestión de Errores y Logging

El módulo `logging` escribe en **errores.log** con formato: `fecha | nivel | mensaje`. Solo se registran `WARNING` y `ERROR` para no saturar el archivo. Los errores de descarga individuales no detienen la cola gracias a `ignoreerrors=True`.

8. Estructura de Archivos

```
■ Proyecto/
■■■ descargar_youtube_pro.py
■■■ generar_manual.py
```

```
■■■ errores.log
■■■ Manual_Tecnico.pdf
■■■ Manual_Usuario.pdf
■■■ .historial/
■   ■■■ descargados_video.txt
■   ■■■ descargados_audio.txt
■■■ descargas/
    ■■■ videos/
    ■■■ audio/
```